Especificação Técnica: Mini-Kernel Simulador

Arquitetura, Ferramentas e Fluxo de Execução

Documentação do Projeto

27 de janeiro de 2026

Sumário

1 Visão Geral **2**

2 Arquitetura Modular (Definição de Ferramentas) **2**

2.1 1. O Loader (`loader.c` / `loader.h`) 2

2.2 2. Gerenciador de Processos (`process.c` / `structures.h`) 2

2.3 3. Gerenciador de Memória (`memory.c` / `memory.h`) 2

3 Especificação das Estruturas de Dados **4**

3.1 Instrução (O Código Carregado) 4

3.2 PCB (Process Control Block) 4

4 Comandos do Console (Kernel) **4**

5 Linguagem dos Programas (.txt) **5**

6 Exemplos de Fluxo de Execução **6**

6.1 Cenário 1: Carregamento e Execução Simples 6

6.2 Cenário 2: Erro de Memória 6

1 Visão Geral

Este documento define a arquitetura técnica para o desenvolvimento de um Mini-Kernel (Simulador de Sistema Operacional). O objetivo é simular o ciclo de vida de processos, gerenciamento de memória e interpretação de comandos, utilizando a linguagem C e estruturas de dados fundamentais.

2 Arquitetura Modular (Definição de Ferramentas)

O sistema deve ser dividido em módulos independentes. Abaixo estão as especificações técnicas para cada componente.

2.1 1. O Loader (`loader.c` / `loader.h`)

Responsável por ler arquivos do disco e transformá-los em estruturas na memória.

- **Entrada:** Nome do arquivo (ex: `"jogo.txt"`).
- **Saída:** Um ponteiro para uma estrutura `PCB` pronta.
- **Ferramentas Obrigatórias:**
  - Leitura de arquivo: `fopen`, `fgets` (leitura linha a linha).
  - Parser (Interpretação): `strtok` ou manipulação de strings para separar *comando* de *argumento*.
  - Estrutura de Armazenamento: **Lista Encadeada Simples** de instruções dentro do PCB.

2.2 2. Gerenciador de Processos (`process.c` / `structures.h`)

Responsável por definir o que é um processo e gerenciar seus estados.

- **Estrutura Principal (PCB):** Deve conter ID, PC (Program Counter), Estado e Ponteiros para o código.
- **Controle de Estado:** Uso obrigatório de `enum` (não usar strings).

```c
typedef enum { NOVO, PRONTO, EXECUTANDO, BLOQUEADO, TERMINADO }
    Estado;

```

- **Organização:** O Kernel deve manter uma **Fila (Queue)** de processos prontos para execução (First-In, First-Out).

2.3 3. Gerenciador de Memória (`memory.c` / `memory.h`)

Responsável por simular a memória RAM física.

- **Estrutura de Dados:** Um vetor de inteiros (`int ram[TAM]`).
- **Representação:**

  - `0`: Espaço livre.
  - `N`: Espaço ocupado pelo Processo ID `N`.

- **Algoritmo de Alocação: First-Fit** (Primeiro Encaixe). O sistema varre o vetor do início e aloca o primeiro buraco contíguo capaz de suportar o tamanho requisitado.

3 Especificação das Estruturas de Dados

As estruturas abaixo devem ser implementadas no arquivo de cabeçalho global (ex: `structures.h`).

3.1 Instrução (O Código Carregado)

Cada linha do arquivo de texto vira um nó desta lista.

```c
typedef struct NoInstrucao {
    char comando[20];        // Ex: "malloc", "print"
    int arg_numerico;        // Ex: 1024
    char arg_texto[100];     // Ex: "Ola Mundo"
    struct NoInstrucao *prox;
} NoInstrucao;
```

3.2 PCB (Process Control Block)

A ficha técnica do processo.

```c
typedef struct PCB {
    int id;                  // Identificador unico (PID)
    char nome[50];           // Nome do programa
    int pc;                  // Contador de linha (apenas visual)
    Estado estado;           // Enum (PRONTO, EXECUTANDO...)

    // O Codigo do Programa
    NoInstrucao *inicio_instrucoes;  // Inicio da lista (Ancora)
    NoInstrucao *instrucao_atual;    // Onde a CPU parou (Cursor)

    struct PCB *prox;        // Proximo processo na fila de prontos
} PCB;
```

4 Comandos do Console (Kernel)

Estes comandos são digitados pelo usuário no terminal do simulador.

Comando	Exemplo	Ação do Sistema
`load`	`load jogo.txt`	Lê o arquivo, cria o PCB, carrega as instruções na lista e coloca o processo na fila de PRONTO.
`execute`	`execute`	Roda o escalonador (Round Robin ou FIFO). Executa instruções do processo da vez.
`mem`	`mem`	Imprime o mapa visual do vetor RAM (ex: `[1][1][0][0]`).
`ps`	`ps`	Lista PID, Nome e Estado atual de todos os processos.
`kill`	`kill 1`	Força a remoção do processo ID 1 da fila e limpa sua memória.
`exit`	`exit`	Encerra o simulador.

5 Linguagem dos Programas (.txt)

Os arquivos de texto devem seguir o padrão: `COMANDO ARGUMENTO`.

Comando	**Sintaxe**	**Efeito no Simulador**
`malloc`	`malloc 5`	Solicita 5 espaços contíguos na RAM. Se falhar, o processo deve ser abortado.
`print`	`print Ola`	Imprime a mensagem na tela.
`wait`	`wait 3`	O processo sai da CPU e fica BLOQUEADO por 3 ciclos.
`exit`	`exit`	O processo termina voluntariamente e libera sua memória.

6 Exemplos de Fluxo de Execução

6.1 Cenário 1: Carregamento e Execução Simples

Passo 1: Arquivo criado (`teste.txt`)

```
print Iniciando
malloc 3
print Alocado
exit
```

Passo 2: Usuário digita no Kernel

```
> load teste.txt
[KERNEL] Processo carregado com PID 1.
```

O que acontece internamente: O Loader lê o arquivo, cria a lista encadeada com 4 nós e coloca o PCB na fila com estado PRONTO.

Passo 3: Usuário executa

```
> execute
[PID 1] DISSE: Iniciando
[KERNEL] Memoria alocada para PID 1 (3 slots).
[PID 1] DISSE: Alocado
[KERNEL] Processo 1 finalizado. Memoria liberada.
```

O que acontece internamente: O escalonador pega o PID 1. Executa instrução por instrução. Ao chegar no `malloc`, preenche o vetor RAM. Ao chegar no `exit`, limpa o vetor RAM e apaga o PCB.

6.2 Cenário 2: Erro de Memória

Passo 1: Estado Inicial Suponha que a RAM tem tamanho 10 e já tem 8 espaços ocupados.

Passo 2: Programa (`grande.txt`)

```
malloc 5
```

Passo 3: Execução

```
> load grande.txt
> execute
[KERNEL] ERRO: Processo 2 tentou alocar 5 slots, mas nao ha espaco contiguo.
[KERNEL] Processo 2 abortado.
```